```
str2 = ""
str2 << 1 << 2 << 3   # => "\u0001\u0002\u0003"
[1, 2, 3].pack("c*")  # => "\x01\x02\x03"
```

The first example here is using escape sequences to put raw bytes into the string, the second example is using the shovel operator to add the numbers one by one, and the third is using Array#pack, the argument c* says that all the elements of the array should be converted as 8-bit unsigned values.

Finding Files

Ruby has a couple of utility methods that can help you find files. Typically, when you search for a file, the pathname is relative to the directory from where the script was invoked, but in a large code base that's unlikely to be the file you are writing code in.

To help orient yourself, Ruby provides __FILE__, which always has the relative name of the file it is contained in, and __dir__ which has the absolute pathname of that file. File.realpath returns the absolute path to a file, so File.realpath(__FILE__) gives you the absolute path to the current file, and so __dir__ = File.dirname(File.realpath(__FILE__)).

You are probably wondering why __FILE__ is capitalized, while __dir__ is not. Technically, __FILE__ is a reserved word (not quite a constant, but close), while __dir__ is a method of Kernel. That's not much of an explanation, but it's what we've got.

One use of realpath that you might do is to use it to figure out what Ruby is thinking of as the base path. You might try to do something like File.open("local.txt"), only to have Ruby tell you that local.txt doesn't exist. In that case, putting in a debug statement like puts File.realpath("local.txt") will go a long way toward orienting you as to where Ruby thinks it's looking.

I/O with Streams

Just as you can append an object to an Array using the << operator, you can also append an object to an output IO stream:

```
endl = "\n"
$stdout << 99 << " red balloons" << endl
```

produces:

```
99 red balloons
```

Again, the << method uses to_s to convert its arguments to strings before printing them.

There are actually some good reasons for using the << operator. Because other classes (such as String and Array) also implement a << operator with similar semantics, you can quite often write code that appends to something using << without caring whether it is added to an array, a file, or a string. This kind of flexibility also makes unit testing easy. We discuss this idea in greater detail in Chapter 21, Ruby Style, on page 345.

Doing I/O with Strings

There are often times where you need to work with code that assumes it's reading from or writing to one or more files. But you have a problem: the data isn't in files. Perhaps it's available instead via a remote network call, or it has been passed to you as command-line parameters. Or maybe you're running unit tests, and you don't want to alter the real file system.